AI 비서 미션 평가 질문·답변

작성·대조일: 2026-10-27 (Asia/Seoul)

대상: jec5500-prog/M1-2-codyssey, main

어떤 질문지를 채택했나

이 문서는 앞서 확인한 **AI 비서 평가 질문 내용**을 대상으로 한다. 배포 URL, Swagger, Firestore CRUD·요약, GPT 채팅·대화 복원, FastAPI/Pydantic 설계, 컨텍스트 주입, CORS·콜드스타트에 관한 질문이 이 프로젝트의 원본 과제와 직접 대응한다. 이미지 파일명이나 저장 폴더명만으로 과제 소속을 판단하지 않는다.

현재 C:/Users/josep/Desktop/M1-1-Project/질문.webp에는 **커피 선물 시계열 분석** 질문지가 저장돼 있으며, 그 질문은 M1-1 평가 답변에서 다룬다. 화면에 나온 다른 제출자의 저장소 주소나 과거 FAIL 표시를 이 AI 비서 프로젝트의 평가 결과로 해석하지 않는다.

평가 전 확인한 실제 상태

GitHub·Vercel

GitHub 원격 저장소는 jec5500-prog/M1-2-codyssey이다. Vercel 프로젝트 Overview에서 main의 Production **Ready**와 제출 도메인 연결을 확인했다. 문서 갱신 후에는 최신 커밋과 Ready 상태를 다시 확인한다.

Render·Swagger

백엔드는 {"status":"ok"}를 반환했고, /openapi.json은 OpenAPI 3.1.0이었다. Swagger UI와 실제 CRUD·요약 검증은 API 검사 기록에 있다.

Firestore 데이터

data에 서울 일평균기온 122건(2026-06-01~09-30), 평균 25.1197℃. 배포 API와 독립 MCP 클라이언트의 건수·통계가 일치했다. 출처와 정제 규칙, 배포 화면.

AI·대화

실제 GPT 응답, get_data_summary 도구 호출, 자동 저장·재조회·임시 대화 삭제는 실행 기록과 최신 배포 채팅 화면으로 확인했다. 오류 때 성공으로 간주하지 않았다.

회귀 검사

이번 대조에서 Python 테스트 **64개**, 프론트 Node 테스트 **8개** 통과. 실서비스 CRUD 15항목은 별도 기록, 상세 요구사항 대조는 인수 검사표에 있다.

접속 조건

Vercel 프론트는 소유자가 선택한 Require Log In: All Deployments 설정을 유지한다. **로그인된 권한 있는 브라우저에서는 앱이 작동하지만, 평가자가 혼자 비로그인 URL을 열면 Vercel 로그인 화면이 나온다.** 따라서 '누구나 URL만으로 접속' 항목은 무조건 통과라고 말할 수 없다. 현장에서는 소유자의 로그인 세션으로 시연해야 한다. 백엔드·Swagger는 공개 접속 가능하다.

아래 답변은 **질문지의 질문을 이 프로젝트에 맞춰 재작성**한 발표용 예시다. 제출 직전 수치와 배포 상태는 다시 읽고, 변했다면 현행 값을 말한다.

> **주석 읽는 법:** 각 답변 뒤의 '발표자 주석'은 답변에 덧붙일 말과 시연 주의점이다. 평가자가 묻는 범위에 맞춰 필요한 부분만 설명한다. 주석의 시연 제안은 이미 수행한 검증 결과와 구분하고, 현장에서 실제로 확인한 값으로 말한다.

항목 1 — 작동과 시연

Q1. 배포 URL로 접속할 수 있나요?

A. 프론트는 Vercel 제출 주소, API는 Render 주소입니다. Vercel Overview의 Production Ready와 로그인된 실제 주소에서 데이터·채팅을 확인했습니다. 다만 프론트는 Vercel 팀 로그인 보호가 켜져 있습니다. 심사자가 단독으로 비로그인 접속하면 앱이 아니라 로그인 화면을 보므로, 현장에서는 제 권한 있는 로그인 세션으로 보여드리겠습니다. '공개 링크 단독 접속 가능'이라고 주장하지 않습니다.

발표자 주석: 시작 전에 로그인된 브라우저와 Vercel Overview의 main / Ready를 준비하세요. 심사자가 자신의 브라우저에서 URL을 직접 열어야 한다고 하면 이 항목은 보호 설정 때문에 조건부라고 먼저 말해야 합니다.

Q2. Swagger UI는 어디에서 볼 수 있나요?

A. Render /docs입니다. FastAPI가 backend/main.py의 라우터와 backend/models.py의 Pydantic 스키마로 OpenAPI 문서를 생성합니다. /openapi.json의 응답과 실제 API 검사를 확인했습니다. Render 무료 인스턴스는 유휴 후 첫 요청이 지연될 수 있습니다.

발표자 주석: 시연 직전에 /health를 먼저 열어 Render를 깨우면 대기 시간을 줄일 수 있습니다. Swagger에서는 우선 읽기 전용 GET /api/data/summary를 실행해 보세요. POST /api/chat은 실제 API 비용과 대화 저장을 수반합니다.

Q3. 웹에서 데이터를 추가하면 Firestore와 화면에 반영되나요?

A. 날짜·값·메모를 입력하면 프론트가 POST /api/data를 호출하고, 백엔드가 Firestore data 문서로 저장합니다. 성공 후 목록과 /api/data/summary를 다시 불러와 표·요약·그래프를 갱신합니다. 수정은 PUT, 삭제는 DELETE입니다. 실서비스에서 임시 행을 추가→조회→수정→삭제하고 기본 122건으로 복귀한 검사 기록이 있습니다. 구현은 frontend/app.js, backend/routers/data.py입니다.

발표자 주석: 현장에서는 평가 시연용 같은 메모의 임시 데이터 1건으로 건수 변화를 보여주고, 마지막에 반드시 삭제해 122건으로 되돌리세요. 브라우저가 Firebase에 직접 쓰는 구조가 아니라 **프론트 → FastAPI → Firestore** 순서입니다.

Q4. /api/data/summary와 AI 답변은 실제 데이터를 반영하나요?

A. 이 API는 날짜순 데이터의 건수·기간·평균·합계·최소·최대·최신값·최근 7건 추세를 계산합니다. 확인 당시 122건, 평균 25.1197℃였습니다. 채팅은 **같은**

compute_summary 함수로 요청 시점의 Firestore 데이터를 다시 요약해 시스템 메시지에 넣습니다. GPT가 더 구체적인 기간 비교를 원하면 읽기 도구를 호출합니다. 실제 답변에서 전체 122건·약 25.1℃와 6월·9월 평균이 API 값과 일치한 증거가 있습니다. 구현은 backend/services/data_summary.py, backend/services/ai_chat.py입니다.

> **발표자 주석:** '최근값'은 오늘(10월 27일)의 기온이 아니라 **저장된 데이터의 마지막 날짜인 9월 30일 값**입니다. '최근 추세'도 최근 7일이 아니라 날짜순 마지막 **7건**의 추세입니다. ERA5는 재분석 자료로, 관측소 실측이라고 소개하지 마세요.

Q5. 대화 저장·목록·불러오기는 어떻게 확인하나요?

A. POST /api/chat이 성공적인 AI 답변을 받은 뒤 사용자 질문과 답변을 같은 conversations 문서의 messages에 자동 추가하고 conversation_id를 돌려줍니다. 프론트는 GET /api/conversations로 목록을 새로 읽고, 선택하면 GET /api/conversations/{id}의 전체 메시지를 재표시합니다. 수동 저장 POST /api/conversations와 삭제 API도 제공합니다. API 검사와 복원 화면이 있습니다.

> **발표자 주석:** 화면의 기본 흐름은 **AI 답변 후 자동 저장**입니다. 별도의 '수동 저장' 버튼이 있는 것처럼 설명하지 마세요. 수동 저장은 Swagger에서 확인할 수 있는 API 기능입니다. 답변 실패 화면과 저장된 대화를 혼동하지 않도록 실제 목록을 다시 열어 보여주세요.

Q6. 좁은 화면에서도 사용할 수 있나요?

A. frontend/style.css의 1000px·520px 미디어 쿼리가 사이드바와 주요 패널을 좁은 폭에 맞게 바꾸며, frontend/app.js의 메뉴 버튼으로 사이드바를 열고 닫습니다. 주요 동작은 같은 API·상태 흐름을 사용합니다. 시연 때 브라우저 폭을 줄여 메뉴, 데이터 탭, 채팅 입력을 직접 보여드리겠습니다. 화면 크기별 완전한 접근성 인증까지 받았다는 주장은 하지 않습니다.

발표자 주석: 좁은 화면에서는 사이드바가 메뉴 버튼 뒤로 들어가고 채팅·데이터가 탭으로 전환됩니다. 이번 점검에서 좁은 배포 화면의 요약·차트·추가 폼 표시를 확인했지만, 모든 휴대전화 기종에서의 동작을 보증한다고 말하지는 마세요.

항목 2 — 구조를 설명하는 답변

Q7. FastAPI를 어떤 기준으로 분리했나요?

A. backend/main.py는 앱 생성·CORS·공통 예외 처리·라우터 등록을 맡습니다. backend/routers/data.py는 데이터 HTTP 계약과 Firestore CRUD, backend/routers/chat.py는 채팅·대화 HTTP 계약을 담당합니다. 재사용 계산은 backend/services/data_summary.py, AI 문맥·도구 호출은 backend/services/ai_chat.py, 대화 저장 정책은 backend/services/conversations.py에 둡니다. 데이터 요약을 API와 AI가 똑같은 함수로 계산하게 하려고 분리했습니다.

발표자 주석: 화살표로 main → router → 공용 계산/AI·대화 서비스 → Firestore·OpenAI를 그리면 설명이 쉽습니다. data.py 라우터 안에 Firestore CRUD도 있으므로 모든 DB 코드가 별도 저장소 계층에 있다고 설명하지는 마세요.

Q8. Firestore 컬렉션 구조는 무엇이고 왜 이렇게 했나요?

A. data/{id}는 date(YYYY-MM-DD), value(유한 숫자), memo, 생성·수정 시각을 갖는 시계열 1건입니다. 날짜로 정렬·기간 필터를 하려고 포인트별 문서로 저장했습니다. conversations/{id}는 title, created_at, updated_at, messages[]를 갖습니다. 메시지에는 role, content, timestamp, tools_used를 저장하여 한 대화를 한 번에 복원할 수 있습니다. 200메시지·900KB 상한과 버전 조건부 갱신을 적용했습니다. 장기·대규모 대화라면 메시지를 별도 하위 컬렉션으로 분리할 수 있습니다.

발표자 주석: data는 **포인트 1건=문서 1개**, conversations는 **대화 1개=문서 1개**라는 대비가 핵심입니다. 데이터 API가 새로 만드는 문서는 자동 ID를 쓰고, 기본 122건 적재 스크립트는 재실행 중복을 피하려고 결정적 ID를 씁니다. 현재 대화는 사용자별로 분리되지 않은 시연용 컬렉션입니다.

Q9. Pydantic 스키마와 검증 기준은 무엇인가요?

A. backend/models.py에서 생성·부분수정·조회·요약·채팅·대화 스키마를 분리했습니다. 데이터 날짜는 형식뿐 아니라 실제 존재하는 날짜인지 확인하고, 값은 FiniteFloat로 NaN/Infinity를 거절하며 범위를 제한합니다. 메모 500자, 채팅 1~4000자와 공백 제거, 대화 메시지 역할·길이·개수를 검증합니다. 기간의 시작일>종료일과 목록 limit도 검증하며 잘못된 요청은 422로 돌려줘 DB와 GPT에 오염된 입력이 도달할 가능성을 줄입니다.

발표자 주석: 2026-02-30 같은 존재하지 않는 날짜나 NaN을 예로 들면 **형식 검사와 의미 검사**의 차이가 드러납니다. 입력 검증은 부정확한 자료를 줄여 주지만, 기온 데이터 자체가 실제로 옳은지까지 보증하지는 않습니다.

Q10. 프론트의 상태 흐름은 어떻게 연결했나요?

A. frontend/app.js의 state에 현재 conversationId, 대화 목록, 데이터·요약, 수정 대상, 전송·불러오기 중 상태를 둡니다. 데이터 CRUD가 성공하면 목록·요약을 다시 읽어 화면을 일치시키고, 채팅이 성공하면 받은 conversation_id를 현재 대화로 설정한 뒤 목록을 갱신합니다. 이전 대화를 선택하면 상세 메시지를 가져오고, conversationLoadToken으로 늦게 도착한 이전 요청이 최신 선택을 덮어쓰지 않게 합니다. 새 대화 버튼은 현재 ID를 비웁니다. 프론트 회귀 검사가 비동기 선택 경합을 확인합니다.

발표자 주석: '새 대화'는 저장된 기록을 삭제하는 기능이 아니라 **현재 선택을 초기화**하는 기능입니다. 이전 대화를 불러오는 동안 다른 대화를 고르면 늦은 응답이 새 선택을 덮지 않도록 토큰을 쓰는 이유를 예로 들어 설명하세요.

항목 3 — 설계 판단

Q11. 왜 '요약→시스템 프롬프트' 컨텍스트 주입을 썼나요? 장단점은요?

A. 일반 GPT는 개인 Firestore 데이터를 모르므로 매 질문마다 저장된 최신 통계를 계산해 시스템 메시지에 넣습니다. 이 방식은 모델이 기간·건수·평균 같은 공통 사실을 바로 참조하게 하고, 사용자가 데이터를 수정한 뒤 새 질문하면 최신 값을 반영합니다. 장점은 구현이 단순하고 근거를 공용 계산 함수로 통일하기 쉽다는 점입니다. 단점은 요약이 원본 전체의 모든 세부 패턴을 담지 못하고 토큰을 쓰며, 모델이 여전히 해석을 잘못할 수 있다는 점입니다. 그래서 기간 비교·최근 원본·극값에는 Function Calling으로 다시 조회하고, 원본 memo의 지시는 따르지 않도록 프롬프트에 명시했습니다.

> **발표자 주석:** '모델이 내 데이터를 학습했다'가 아니라 **질문할 때 현재 요약을 문맥으로 전달한다**고 표현하세요. 요약과 도구 조회는 답변의 근거를 높이지만, 모델의 모든 문장을 참으로 보증하지는 않습니다.

Q12. 요약 API를 채팅과 별도로 둔 이유는 무엇인가요?

A. /api/data/summary는 AI 호출 없이 통계만 반환하므로 대시보드·테스트·MCP 클라이언트가 재사용할 수 있습니다. 채팅 내부도 동일한 compute_summary를 호출해 API와 같은 계산식을 사용합니다. 따라서 화면 표시와 AI 문맥의 수치가 어긋나는 일을 줄이고, OpenAI 키나 크레딧 상태와 무관하게 데이터 기능을 검증할 수 있습니다.

> **발표자 주석:** Swagger에서 요약 API만 먼저 실행해 결과를 보여주면 AI 답변과 통계 계산의 책임이 분리됐다는 점이 보입니다. 이 호출은 OpenAI 비용은 들지 않지만 Firestore 조회는 수행합니다.

Q13. 대화는 언제, 어떤 형태로 저장하나요?

A. 자동 대화는 GPT의 비어 있지 않은 답변을 얻은 뒤 질문·답변 두 메시지를 함께 추가합니다. 실패한 질문을 정상 답변처럼 저장하지 않습니다. 메시지의

역할·본문·시각과 사용된 도구명을 보존하고, conversation_id로 같은 대화에 이어 씁니다. 수동 대화 저장 API는 평가 요구사항을 충족하기 위해 별도로 제공합니다. 저장 한도를 호출 전에 검사하고, 동시 갱신에는 Firestore 업데이트 시각 조건과 제한된 재시도를 사용합니다.

> **발표자 주석:** 화면에서 오류 문구가 보였다고 해서 DB에 '성공한 AI 답변'으로 저장된 것은 아닙니다. 자동 저장 성공 여부는 응답의 conversation_id로 목록·상세를 다시 읽어 확인해야 합니다. 저장 도중 시간 초과가 나면 같은 질문을 바로 반복하기 전에 기록부터 확인하세요.

Q14. 환경변수를 왜 쓰며 실제로 어디에 적용했나요?

A. OPENAI_API_KEY와 FIREBASE_SERVICE_ACCOUNT_JSON은 백엔드 로컬 .env 또는 Render 비밀 환경변수에만 둡니다. 저장소에는 .env.example만 올리고 실제 키는 Git에서 제외합니다. ALLOWED_ORIGINS는 Render의 허용 프론트 주소, API_BASE_URL은 Vercel 빌드 시 프론트에 기록되는 **공개 API 주소**입니다. 브라우저에 비밀 키를 넣지 않습니다. Vercel 설정 스크립트는 HTTPS origin인지 검사합니다. 이 분리로 개발·배포 주소를 바꾸면서 키 노출을 막습니다.

> **발표자 주석:** 평가 화면에서는 **환경변수 이름과 역할만** 보여주고 실제 키 값·서비스 계정 JSON은 열지 마세요. API_BASE_URL은 공개해도 되는 서버 주소지만, OPENAI_API_KEY와 Firebase 자격증명은 서버 전용이라는 차이를 강조하세요.

항목 4 — 장애와 확장

Q15. Render 첫 접속 지연을 어떻게 다뤘나요?

A. Render 무료 서비스의 유휴 후 기동을 고려해 요청이 2.5초 이상 걸리면 프론트에 서버 준비 안내 배너가 나타납니다. GET 요청은 502/503/504나 네트워크 오류에 최대 두 번 재시도하고, 타임아웃·오류를 사용자에게 알립니다. POST는 중복 저장·중복 과금 위험 때문에 자동 재시도를 하지 않습니다. UI 로

딩 표시와 90초 API 기본 제한, 120초 채팅 대기 제한을 둡니다. 그래도 무료 서비스의 실제 기동 시간이 길면 첫 요청이 실패할 수 있어 다시 시도하도록안내합니다.

> **발표자 주석:** '콜드스타트를 없앴다'가 아니라 **기다리는 이유를 안내하고 읽기 요청을 제한적으로 재시도한다**고 설명하세요. 채팅·데이터 추가 같은 쓰기 요청을 자동 반복하면 중복 저장이나 과금이 생길 수 있어 재시도 정책이 다릅니다.

Q16. CORS 문제와 해결 방식은 무엇인가요?

A. Vercel 프론트와 Render API의 origin이 달라 브라우저가 교차 출처 정책과 preflight를 적용합니다. backend/main.py의 CORSMiddleware에 ALLOWED_ORIGINS로 로컬 개발 주소 및 제출용 Vercel origin을 정확히 등록하고, 필요한 메서드·헤더를 허용했습니다. 자격증명 쿠키 허용은 꺼 두었습니다. Render에 배포된 origin에서 preflight 200을 확인했습니다. 이것은 브라우저 통신 설정이며 사용자 인증은 아닙니다.

> **발표자 주석:** origin은 https://m1-2-codyssey-frontend.vercel.app처럼 **스킴+호스트** 단위입니다. CORS 허용을 해도 직접 API를 호출하는 외부 클라이언트를 막는 인증 수단은 아닙니다. Vercel 로그인 보호도 프론트 배포에 적용될 뿐 Render API 보호와는 다릅니다.

Q17. 사용자 입력을 그대로 저장·전달할 때 위험과 최소 대응은요?

A. 악성 프롬프트가 memo나 질문에 섞이면 답변을 왜곡할 수 있고, HTML/CSV 인젝션·데이터 오염·GPT 비용 증가가 발생할 수 있습니다. 날짜·숫자·길이·기간을 Pydantic으로 검증하고, 프론트 채팅 표시는 HTML 이스케이프합니다. CSV는 수식 시작 문자를 중립화합니다. 프롬프트는 Firestore 메모·도구 결과를 신뢰할 수 없는 데이터라고 명시합니다. GPT는 IP당 분당 6질문, 출력 600토큰·도구 횟수·전체 처리 시간을 제한합니다. **공개 API에 사용자별 인증·소유권 분리는 아직 없으므로** 실제 개인정보나 비공개 대화는 넣지 않는 시연용 범위입니다.

> **발표자 주석:** 현재 제한은 비용·오남용을 줄이는 **최소 대응**이지 완전한 방어가 아닙니다. 특히 IP별 요청 제한은 단일 프로세스 메모리 기준이고 사용자 인증을 대신하지 않습니다. 실제 다중 사용자 서비스라면 인증, 사용자별 Firestore 접근 통제와 서버 측 예산 관리가 추가로 필요합니다.

Q18. 데이터가 늘거나 '최근 30일' 기준으로 바뀌면 어떻게 확장하나요?

A. 현재 fetch_points(start,end,limit)가 날짜 필터·정렬을 지원하고 /api/data/summary에도 시작·끝 날짜를 받을 수 있습니다. 우선 최근 30일의 날짜 범위를 서버에서 계산해 이 공용 경로로 넘기고, 대시보드·AI가 같은 범위를 쓰도록 계산 기준을 명시하겠습니다. 규모가 커지면 전체 문서를 매번 읽는 현재 방식은 비용·지연이 늘므로 기간별 집계 문서나 캐시/증분 갱신을 도입하고, 커서 페이지네이션·색인·결측/중복 정책을 추가하겠습니다. UI에는 '전체'와 '최근 30일'을 구분해 표시하겠습니다.

> **발표자 주석:** 먼저 '최근 30일'을 **오늘 기준**으로 볼지 **저장된 마지막 날짜 기준**으로 볼지 정해야 합니다. 현재 자료는 9월 30일까지라 10월 27일 기준 최근 30일은 데이터가 거의 없을 수 있습니다. 이 선택이 UI·요약·AI 답변에 모두 동일하게 적용돼야 합니다.

항목 5 — 보너스와 시연 순서

이 질문지의 보너스 항목은 세부 문항보다 문제 해결 설명에 가깝다. 원본 미션의 선택 과제는 **Function Calling + MCP**, 확장 지표·그래프·CSV·다크 모드이다. 실제 get_data_summary 도구 호출과 외부 MCP stdio 클라이언트의 list_tools → call_tool 검증, 합계·최근 7건 추세, 선 그래프, CSV 내보내기, 다크 모드가 구현되어 있다. 호출 근거, MCP 검사, README 설명을 보여준다. GPT Actions는 MCP와 **택일**하는 원본 요건이므로 구축했다고 주장하지 않는다.

> **발표자 주석:** 보너스는 '둘 다 설치했다'가 아니라 **GPT Function Calling과 외부 MCP 클라이언트 연동을 검증했다**고 설명하세요. 문제 해결 사례로는 오래 걸리는 Render 첫 요청 안내, Vercel–Render CORS, 대화 선택의 비동기 경합을 짧게 들어도 좋습니다.

평가 자리에서는 다음 순서로 시연한다.

1. 권한 있는 Vercel 로그인 브라우저에서 제출 URL을 열고 data 122건·평균·차트를 보여준다. 새 배포 직후에는 Overview에서 **Ready / main**을 먼저 확인한다.
2. Swagger에서 /api/data/summary, /api/chat, /api/conversations를 찾아 구조를 보여준다.
3. 데이터 한 건을 추가해 목록·요약 변화를 보여주고, 같은 행을 수정·삭제해 원래 122건으로 되돌린다. 공개 시연 데이터이므로 개인 정보를 입력하지 않는다.
4. 채팅에서 데이터 기간·평균을 질문하고, 답변·도구 사용 표시·자동 저장된 목록을 확인한 뒤 이전 대화를 다시 연다. 실시간 OpenAI 장애 시에는 성공했다고 말하지 말고 기존 실제 응답 기록을 증거로 제시한다.
5. 브라우저 폭을 줄여 메뉴·채팅·데이터 탭을 보여주고, 필요하면 CSV·다크 모드·MCP 검사 기록을 설명한다.

완료 판정의 경계: 원본 미션의 기능·기술·보너스 구현과 로그인된 배포 시연은 근거가 있다. 다만 **평가자가 자신의 비로그인 브라우저에서 프론트 URL에 직접 접속해야 한다는 해석**이라면 Vercel 보호 설정 때문에 해당 접속 항목은 충족되지 않는다. 이 설정은 사용자가 유지하기로 선택했으므로 평가 방식에 맞춰 로그인 시연을 준비해야 한다. 평가 결과 자체를 미리 PASS라고 단정하지 않는다.